Exercise : Advanced Classes

Step 1 : Employee class

Make an `Employee` class representing some employee working in a company in the python module `employee.py`. The `Employee` class should have two attributes representing the name and the ID of an employee.

Add a constructor to the `Employee` class which sets the attributes when they are passed. Make a simple program which declares an `Employee` object and display the values of the attributes of the `Employee` object by accessing the attributes directly.

Use code like the following :

```
class Employee(object) :
    def __init__(self, name):
        print("Calling Employee constructor")
        self.name = name
```

Next make the attributes private in the Employee class and add public set and get member functions in the Employee class to be able to access these attributes. Change the values of the attributes through the accessor functions and display their changed values. Use code like the following :

```
    def set_name(self, value):
        self.__name = value
    def del_name(self):
        del self.__name
```

Notice that private data cannot by accessed directly from outside of the class. This demonstrates the object oriented principle of encapsulation. In Python there is a workaround so you can still access private data. Use this workaround also for demonstration purpose.

Also add a destructor to the class and keep a class variable which counts how many instances of the class are present. How would you implement the uniqueness of the ID variable in relation to the variable the count for the number of instances? Use code like the following :

```
    def __del__(self):
        class_name = self.__class__.__name__
        print class_name, "destroyed"
```

Write print statements when the constructor or destructors and the getters and setters are called. Also print an instance variable directly. What is the result?

Next make a property in the class with the following code :

```
  name = property(get_name, set_name, del_name, "name's docstring")
```

Access the `Employee` object through the property as follows :

```
    e1.name = "Some Name"
    print(e1.name)
```

Also write statements to show the class and object data :

```
print("Employee.__doc__:", Employee.__doc__)
print("Employee.__name__:", Employee.__name__)
print("Employee.__module__:", Employee.__module__)
print("Employee.__bases__:", Employee.__bases__)
print("Employee.__dict__:", Employee.__dict__)
```

Put the Employee class in a script in the employees package and add code to be able to export the Employee class from this package.

Give the Employee class a `calcSal()` method as follows :

```
def calcSal(self):
    return 0
```

Optionally you can also add a `hiringdate` attribute to the `Employee` class.

Add the following code to test the `Employee` class in stand alone mode :

```
if __name__ == '__main__':
    main()
```

Step 2 : Company Script

Make a Company script above the `employees` package.

Add the necessary import statements to be able to find the `Employee` class in the `employee` script in the `employees` package. If you keep the __int__.py file in the package empty your code could look as follows :

```
import employees.employee

e1 = employees.employee.Employee("Albert Einstein")
```

Create a number of employees and put them in a list, called for instance `employeeList`. Next iterate through the list and print the name, id and salary of the employees in the list.

If time permits :

Alternatively you could create a `Company` class with functions to add or to remove employees from the `employeeList` which displays a list of employees including their name, ID. In that case make a simple script which declares a `Company` object, add several employees to the company and display a list of employees.

The relationship between the `Company` class and the `Employee` class is an association relation of the normal type. We can say every `Company` employs `Employees` or every `Employee` works for a `Company`.

Step 3 : Inheritance

Derive a `WageEmployee` class from the `Employee` class. The `WageEmployee` class inherits the attributes and public methods of the Employee class. Define two additional attributes in `WageEmployee` `wage`, representing the hourly wage, and `hours`, representing the hours worked. Both these attributes should be private. Also implement get, set and del methods for these attributes. Your code could look as follows :

```
    def get_wage(self):
        return self.__wage
    def get_hours(self):
        return self.__hours
    def set_wage(self, value):
        self.__wage = value
    def set_hours(self, value):
        self.__hours = value
    def del_wage(self):
        del self.__wage
    def del_hours(self):
        del self.__hours
```

Put the `WageEmployee` in the same package as the `Employee`.

Further define a `calcSal` method in `WageEmployee` which returns the salary of a `WageEmployee` by multiplicating `hours` and `wage`. Your code could look as follows :

```
    def calcSal(self):
        return self.__wage * self.__hours
```

De relationship between a `WageEmployee` and an `Employee` is a generalization / specialization, also called an inheritance relation. A `WageEmployee` is a special type of `Employee`.

Add a constructor to the `WageEmployee` class which accepts type the name and id attributes. Use this constructor also to set the hours and wage attributes to default values. Notice that in order to fill the name and id attribute an appropriate `Employee` constructor must be called. It is not possible to access the attributes directly. This constructor should already be present in the `Employee` class. Call this constructor from the `WageEmployee` constructor. Use code like the following :

```
class WageEmployee(Employee) :
    def __init__(self, name, wage, hours):
        super(WageEmployee,self).__init__(name)
        print("Calling WageEmployee constructor")
        self.__wage = wage
```

```
        self.__hours = hours
```

Also make `wage` and `hours` properties as follows :

```
    wage = property(get_wage, set_wage, del_wage, "wage's docstring")
    hours = property(get_hours, set_hours, del_hours, "hours's docstring")
```

Next declare an instance of a `WageEmployee` in the simple program which also declares the `Company` object and call its methods. Or do this in the script if you did not create the `Company` class. Try to use the private name and id attributes from the `WageEmployee` class. You will not succeed. Use the accessor methods instead. Your code could look like the following :

```
w1 = employees.wageemployee.WageEmployee("Werner Heisenberg", 10, 100)
print (w1.name)
print (w1.wage)
print (w1.hours)
```

Add a ManagerEmployee to the Employee hierarchy which is derived Employee and whose salary is a fixed value per month. Add a private monthlySalary attribute of the integer type to the ManagerEmployee class. Its calcSal method should return the value of that attribute. It should be possible to give the ManagerEmployee a salary in its constructor. Add a constructor and add accessor functions for the monthlySalary attribute. Do something like the following :

```
class ManagerEmployee(Employee) :
    def __init__(self, name, weeksal):
        super(ManagerEmployee,self).__init__(name)
        print("Calling ManagerEmployee constructor")
        self.__weeksal = weeksal
```

Put the ManagerEmployee in the same package as the Employee.

Add a ProgrammerEmployee to the Employee hierarchy which is derived from WageEmployee and whose salary consists of the WageEmployee salary + a percentage of the lines produced, minus a percentage of the bugs introduced. Add attributes lines and bugs

Add a constructor and add accessor functions for the lines and bugs attributes.

Conceive an appropriate implementation of its calcSal method

Put the ProgrammerEmployee in the same package as the Employee.

Also add destructors for all these classes.

Step 4 : Polymorphism

Create instances of each of the derived Employee classes in the script which declares the Company object or in the script without the Company object.

Give each employee in the hierarchy, a name, id, and a salary and display these. Next add all the employees to the employeeList in the Company object. Make a function payroll in the Company object

which iterates through the list of employees, calculates each of their salaries, adds them and finally returns the total payroll of the Company.

Calling the calcsal method on the employees in the list, while executing different versions of these calcsal method depending on the concrete derived Employee in the list, is a demonstration of the object oriented principle of polymorphism.

In Python it is sufficient for all classes to have the same method to enable polymorphism. In Java or C++ the calcSal() method must be present in the base class.

If time permits :

Add code that limits the permitted values for the ID attribute to values between 1 and 1000. If someone tries to set the value higher or lower than these limits an error message should be displayed or an exception should be raised and the ID value should be set to the nearest permitted value. The check can be done in the set method and the constructor.

Step 5 : Abstract Base Classes

Create an abstract class Driveable which has a number of abstract methods that are related to driving like start, stop, accelerate, drive and turn.

Notice that you cannot instantiate the class Driveable since it is abstract.

Next create two derived and concrete classes Truck and RaceCar that actually implement the methods from Driveable and which can be instantiated.

Give the methods different implementations and let the Truck and RaceCar drive.